Centro Universitário de Excelência
Sistemas de Informação

MecâniQA–Persistência de dados

Autores: Jadson da Silva Sobrinho
Levi Falcão de Queiroz
Mateus Queiroz Matos Ferreira
Murillo dos Santos Marinho Ferreira

Feira de Santana, 2026

Agenda

Apresentar a persistência em MySQL e a organização das camadas da API.

1. Arquitetura da API:
Evolução do armazenamento e separação de responsabilidades.

2. Persistência com JDBC:
Conexão, comandos SQL e estrutura do banco.

3. Organização dos repositórios:
Interface genérica, Singleton e Template Method.

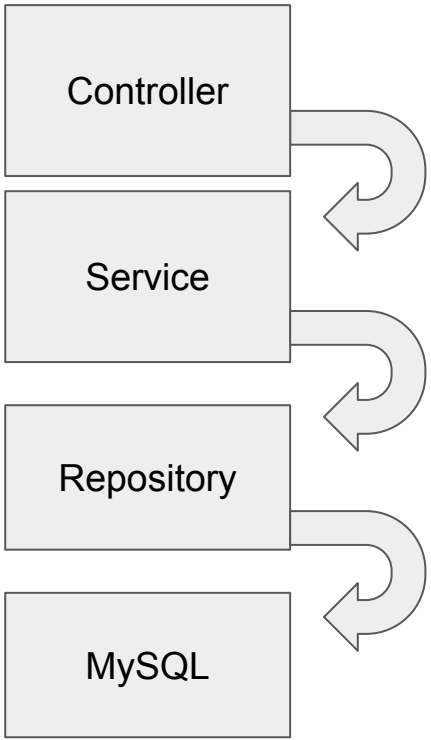
4. Mapeamento dos dados:
Conversão de ResultSet, entidades e DTOs.

5. Validações e exceções:
Regras de negócio e respostas HTTP.

Contexto e arquitetura da API

- A MecâniQA é uma API para gerenciar peças, serviços, pedidos de peças e ordens de serviço de uma oficina mecânica.
- Nas OATs 1 e 2, os dados eram armazenados em listas na memória.
- Na OAT 3, a API utiliza JDBC para gravar e consultar dados no MySQL.

- A camada Service concentra as validações e as regras de negócio.
- Os Controllers recebem DTOs e encaminham as operações aos Services.



Persistência com JDBC e MySQL

- O script schema.sql define tabelas, chaves e restrições.
- Os repositórios possuem SQL manual para INSERT, SELECT, UPDATE e DELETE.

- A ConnectionFactory abre as conexões com o banco.
- O PreparedStatement associa os valores aos parâmetros das consultas.
- As transações utilizam commit e rollback para confirmar ou desfazer gravações.

```
CREATE TABLE IF NOT EXISTS itens_pedido_peca (
 codigo BIGINT PRIMARY KEY AUTO_INCREMENT,
 pedido_codigo BIGINT NOT NULL,
 peca_codigo BIGINT NOT NULL,
 quantidade INT NOT NULL CHECK (quantidade > 0),
```

Organização dos repositórios

- Repository<T, ID> define o contrato comum das operações CRUD.
- T representa a entidade e ID representa o tipo do identificador.

- O Singleton utiliza construtor privado e acesso por getInstance().
- A classe abstrata JdbcSupport reúne operações JDBC comuns.
- O Template Method define a sequência de execução em um método final.

```java
public class PecaRepository extends JdbcSupport implements Repository<Peca, Long> {

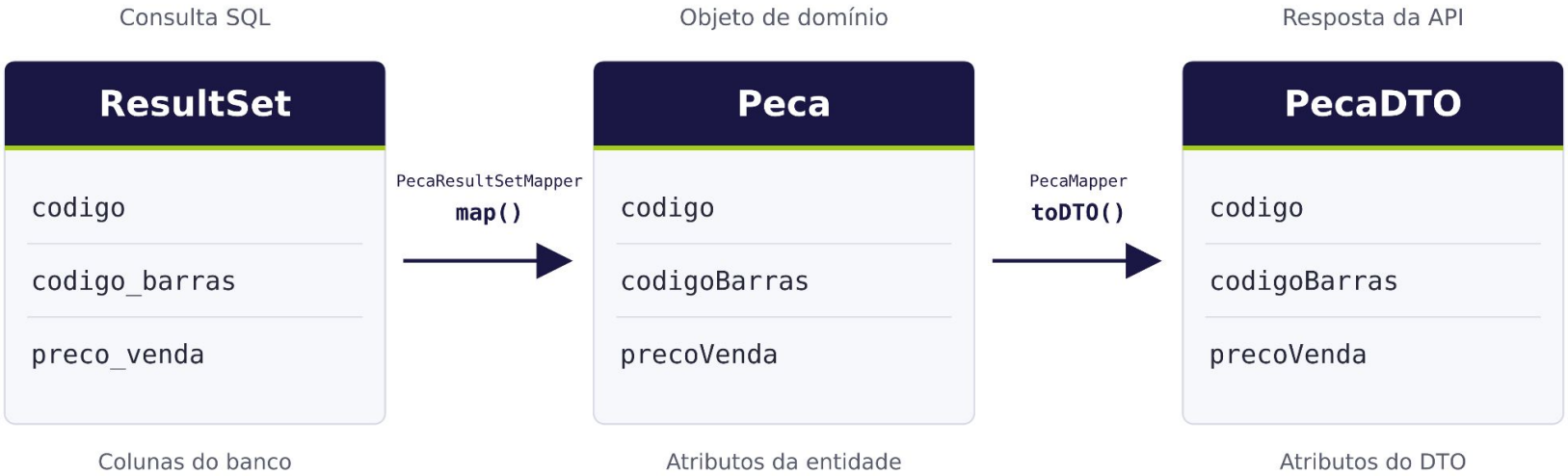
    private static PecaRepository instance;


    private PecaRepository() { super(ConnectionFactory.getInstance()); }


    public static synchronized PecaRepository getInstance() {
        if (instance == null) {
            instance = new PecaRepository();
        }
        return instance;
    }
```

Mapeamento dos dados

- ResultSetMapper transforma os dados da consulta em uma entidade.
- Mapper converte a entidade em DTO para a resposta da API



Services e tratamento de exceções

- O Service valida os dados e executa as regras de negócio.
- Exceções próprias identificam os problemas encontrados.
- O Controller trata as exceções com blocos try/catch.
- O ResponseEntity define o status HTTP e o corpo da resposta.

Situação	Status HTTP
Dados inválidos	**400**
Recurso não encontrado	**404**
Conflito com dados vinculados	**409**
Falha de persistência	**500**

Considerações finais

- A API implementa gravação e recuperação de dados no MySQL.
- A camada Service organiza as validações e as regras de negócio.
- Os repositórios seguem um contrato comum e executam SQL manual.
- Os Mappers realizam as conversões entre banco, entidades e DTOs.
- O projeto inclui coleções Postman e testes de integração.